Walchand College of Engineering, Sangli

(Government Aided Autonomous Institute)

7CS353 Machine Learning Lab — Assignment 8

Decision Tree and Random Forest Classification

Class, Semester	Third Year B. Tech. (CSE), Sem V — AY 2025-26
Linked courses	7CS353 (Lab) and 7CS302 Module IV (Classification)
Builds on	Assignment 7 — classification, model evaluation, confusion matrix
Structure	Part A: Theoretical concepts (not graded) | Part B: Analysis pipeline (15 marks)
Duration	1 lab session (2 hrs)

0. Dataset — Iris, 150 Records

The Iris dataset contains 150 flower samples from three species — Setosa, Versicolor, and Virginica — with four measurements each: sepal length, sepal width, petal length, and petal width (all in cm). It is available directly from sklearn and is the standard benchmark for demonstrating decision trees because the tree structure produced is small enough to visualise and interpret completely.

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.metrics import classification_report, ConfusionMatrixDisplay

iris = load_iris()
X, y = iris.data, iris.target
feature_names = iris.feature_names
class_names   = iris.target_names
print(f"Shape: {X.shape}  Classes: {class_names}")
```

PART A — Theoretical Concepts

Read before starting Part B. Not graded.

A.1 Decision Trees

Structure

A decision tree is a flowchart-like model that partitions the feature space by asking a sequence of binary questions. Each internal node tests a single feature against a threshold; each branch represents one answer; each leaf node holds a class prediction. The depth of the tree — the maximum number of questions from root to leaf — controls model complexity.

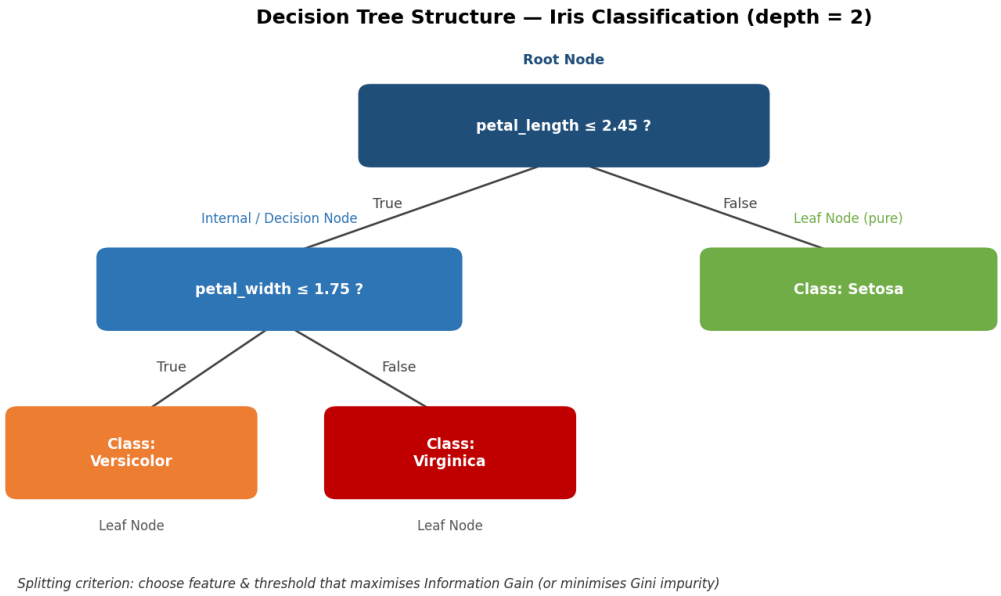


Figure 1: A depth-2 decision tree on the Iris dataset. The root splits on petal_length; the left subtree is already pure (all Setosa). The right subtree needs one further split on petal_width to separate Versicolor and Virginica.

Splitting Criteria

At each node, the algorithm searches for the feature and threshold that best separates the classes. Two common impurity measures are used:

Quantity / Parameter	Formula / Description
Entropy (Information Gain) *Sum over all classes k; p_k = fraction of class k in node S*	$H(S) = -\Sigma\ p_k\ \log_2(p_k)$
Information Gain *Reduction in entropy after a split; choose the split with maximum IG*	$IG = H(parent) - \Sigma\ (\lvert S_v\rvert/\lvert S\rvert) \cdot H(S_v)$
Gini Impurity *Faster to compute; preferred in sklearn default (criterion="gini")*	$G(S) = 1 - \Sigma\ p_k^2$

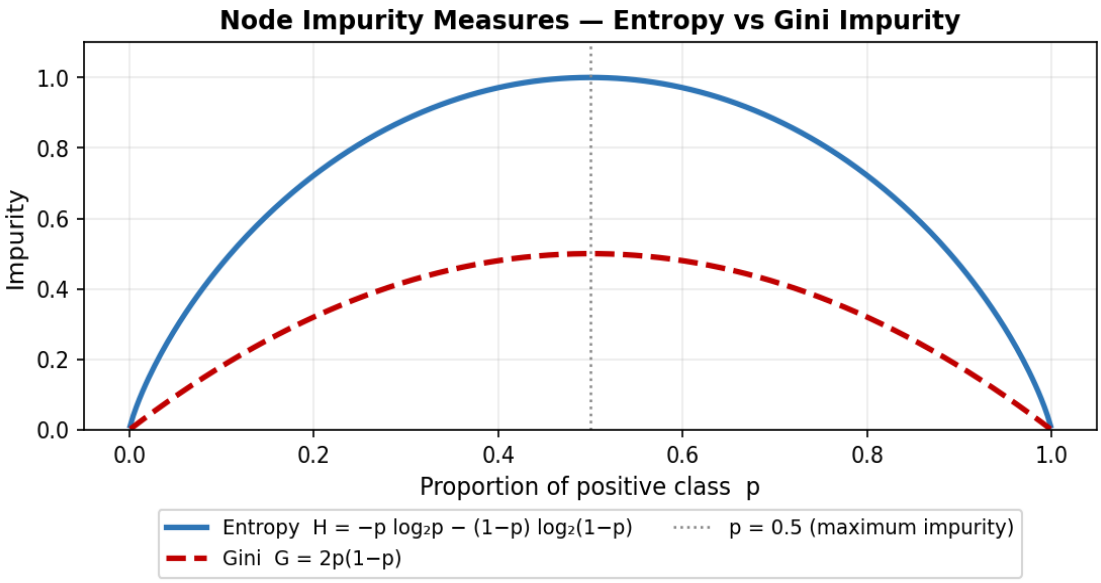


Figure 2: Entropy and Gini impurity as a function of the positive-class proportion p. Both reach maximum at p = 0.5 (equally mixed node) and zero when the node is pure (p = 0 or p = 1). A split is good if it produces child nodes with lower impurity than the parent.

Overfitting and Control

A tree grown to full depth memorises the training data — every leaf becomes a single training example. Key hyperparameters that control complexity:

- max_depth — limits the number of levels. Shallow trees underfit; deep trees overfit.
- min_samples_split — minimum samples required to split an internal node.
- min_samples_leaf — minimum samples required in a leaf node. Higher values smooth the tree.
- Pruning — post-hoc removal of branches that provide little predictive power (cost-complexity pruning, ccp_alpha in sklearn).

A.2 Random Forests

Ensemble Learning — Wisdom of Crowds

A single decision tree is a high-variance model — small changes in training data can produce very different trees. An ensemble combines many trees so that their errors partially cancel. Random Forest achieves this through two sources of randomness:

- Bootstrap aggregating (Bagging): each tree is trained on a different bootstrap sample — n rows drawn with replacement from the training set. About 63% of rows appear in any given sample; the remaining ~37% form the out-of-bag (OOB) set.
- Random feature subsets: at each split, only √p features (for p total features) are considered as candidates. This reduces correlation between trees — trees that disagree with each other are more useful for reducing variance than trees that all make the same mistake.

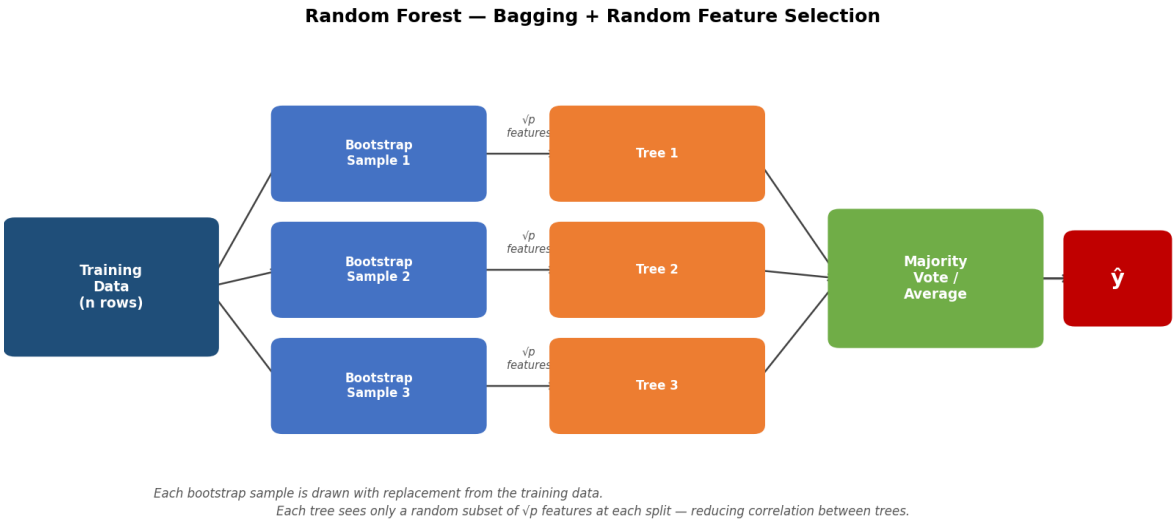


Figure 3: Each bootstrap sample trains one decision tree on a random feature subset. Final predictions are the majority vote (classification) or mean (regression) across all trees.

Out-of-Bag Error and Feature Importance

Out-of-bag (OOB) error. Each training example is OOB for roughly one-third of trees. These trees can be used to generate an unbiased estimate of generalisation error without a separate validation set. Access with oob_score=True in sklearn.

Feature importance. For each feature, sum the total reduction in impurity it causes across all nodes and all trees, weighted by the proportion of samples reaching each node. Normalise to sum to 1. Features with higher importance are better predictors. This is sometimes called Mean Decrease in Impurity (MDI). Note: MDI can overstate importance of high-cardinality features.

A.3 Key Hyperparameters

Hyperparameter	Decision Tree	Random Forest
max_depth	Depth of the tree (default = None: grow until pure leaves)	Same; applied to each individual tree
min_samples_split	Minimum samples to split a node (default = 2)	Same
min_samples_leaf	Minimum samples in a leaf (default = 1)	Same
criterion	"gini" or "entropy" (default = "gini")	Same
n_estimators	N/A — single tree	Number of trees (default = 100); more trees reduce variance
max_features	N/A (uses all features)	Features per split: "sqrt" (default), "log2", or integer
oob_score	N/A	True = compute OOB accuracy without a validation set
ccp_alpha	Cost-complexity pruning; higher = more pruning (default = 0)	N/A — forest averages handle variance

A.4 Decision Tree vs Random Forest

Property	Decision Tree	Random Forest
Model type	Single model	Ensemble of trees

Interpretability	High — can visualise and read the tree	Low — hundreds of trees; no single readable tree
Variance	High — sensitive to training data	Low — averaging reduces variance
Overfitting risk	High (deep trees)	Low — randomness decorrelates trees
Feature importance	Available (single tree)	More reliable (averaged over many trees)
Training speed	Fast	Slower (proportional to n_estimators)
When to use	When interpretability matters; small datasets	When accuracy matters more than explainability

PART B — Analysis Pipeline (15 marks)

Use sklearn for model fitting, tree visualisation, and GridSearchCV. All plots must have a title and labelled axes. Every step in its own cell with a one-line markdown comment.

B1. Load and explore: Load the Iris dataset. Print shape, class distribution, and feature statistics using describe(). Draw a pair plot (scatter matrix) using pd.plotting.scatter_matrix or seaborn pairplot, coloured by class. In a markdown cell, identify which pair of features appears to best separate the three classes.

B2. Impurity from scratch: Using only NumPy, write two functions: entropy(y) that computes $H = -\Sigma\ p_k \log_2(p_k)$, and gini(y) that computes $G = 1 - \Sigma\ p_k^2$. Apply both functions to the full label vector y, then to the subset of labels where petal_length ≤ 2.45, and to the complement. Compute the Information Gain for this split using entropy. In a markdown cell, confirm whether the split identified by petal_length ≤ 2.45 matches the root node of Figure 1 in Part A. Comment on split quality of IG.

B3. Decision Tree — fit and visualise: Perform an 80/20 train-test split (random_state=42). Fit a Decision Tree with max_depth=3 and criterion="entropy". Use sklearn.tree.plot_tree to draw the full tree, showing the feature used, threshold, impurity, and sample counts at each node. Also print export_text(dt) for a text summary. Report classification report and confusion matrix heatmap on the test set.

B4. Depth analysis: Train Decision Trees for max_depth = 1 through 10 and record training accuracy and test accuracy for each. Plot both curves on the same axes (depth on x-axis). In a markdown cell, identify the depth at which the model begins to overfit (training accuracy approaches 1.0 while test accuracy plateaus or drops), and choose an optimal depth with justification.

B5. Random Forest — fit and evaluate: Fit a Random Forest with n_estimators=100, oob_score=True, random_state=42. Print the OOB score and test-set classification report. Draw a horizontal bar chart of feature importances, sorted from highest to lowest. In a markdown cell, compare which features the Random Forest considers most important with what you observed in the pair plot (B1).

B6. Hyperparameter tuning: Use GridSearchCV (cv=5) to tune the Random Forest over: n_estimators ∈ {50,100,200} and max_depth ∈ {None,5,10,15}. Print the best parameter combination and the best cross-validation accuracy. Refit with best parameters and report the final test-set metrics.

B7. Comparison and interpretation: Present a final summary table in markdown:

Model	Test Accuracy	Precision	Recall	F1	
Decision Tree (depth=3)					
Random Forest (default)					

Random Forest (tuned)					

Comment on:

1) Deployment of the model
2) Feature importance
3) Under what circumstances a single decision tree would be preferable to a random forest despite lower accuracy.

Deliverables

- Submit word document as per earlier instructions.
- All plots with titles and labelled axes; unlabelled plots count as incomplete.
- Viva at the start of the next session.

Assessment Rubric (15 marks — Part B only, toward LA2)

Assessed against two Course Outcomes.

CO	Weight	Rubric statement	Very good	Good	Average	Needs improvement	Not done
CO2	10 marks	Fit, visualise, and evaluate Decision Tree and Random Forest; analyse depth vs accuracy; interpret OOB score and feature importance.	All steps complete; tree visualised; depth curve plotted; feature importance sorted and labelled.	Most steps complete; minor omissions in plots or metrics.	B3 or B5 missing; significant gaps in analysis.	Only basic fitting without visualisation or metrics.	Not submitted.
CO3	5 marks	Justify optimal depth, compare models, interpret feature importance, and recommend which model to deploy with evidence.	Depth justified with evidence from curves; model comparison precise; feature importance interpretation insightful.	Comparison and justification present; minor gaps.	Comparison present but unsupported by evidence.	No comparison or justification.	Not done.
Total		**15 marks (CO2 = 10 + CO3 = 5)**					

A step that cannot be explained in the viva does not count as completed.